

TESS light-curve pipeline documentation

This document explains **what the code does, step-by-step**, including **inputs, settings, data filtering, period search, and outputs**. It is written so that a reader who has never seen the code can **recreate the same products** (plots + CSVs + batch metrics) and understand the implementation choices.

0) What this script produces

For each target star, the script:

1. **Searches** for available TESS light curves (prefers **SPOC**, falls back to **QLP**) using Lightkurve.
2. **Downloads** all available sectors (or a subset if you later add selection).
3. **Selects the flux column** based on the downloaded product (**SPOC vs QLP**) and the user-chosen `flux_mode`:
 - o default (requested defaults): **SPOC → PDCSAP, QLP → SAP**
 - o detrended: **SPOC → PDCSAP; QLP → DET_FLUX / KSPSAP_FLUX** (fallback to SAP)
 - o raw: prefer **SAP everywhere** (fallback to detrended choices if SAP is unavailable)
4. **Normalizes** each sector and removes NaNs/Infs.
5. Applies **per-sector filtering** of extreme points using a MAD-based rule.
6. Drops sectors that are clearly bad using a **sector scatter outlier test**.
7. Concatenates remaining sectors and keeps **only cadences with QUALITY == 0** for the Lomb–Scargle analysis.
8. Computes a **generalized Lomb–Scargle periodogram (χ^2 -based)** on a fixed frequency grid (default is `fmin=0.004 1/day` and `fmax=4 1/day`).
9. Picks the best period, estimates an uncertainty from a crude **FWHM** approach, and also considers $2 \times P$.
10. Produces **plots** and a **cleaned_lightcurve.csv** per target as well as periodogram products.

In batch mode, it repeats this for all rows in an input CSV and produces an **augmented CSV** with all per-target metrics.

1) Dependencies and required files

Python packages

- numpy, pandas, matplotlib
 - lightkurve
 - astropy (LombScargle + coordinates + units)
 - scipy (find_peaks)
 - astroquery (Gaia TAP)
 - decimal (robust parsing of integer-like IDs that may appear as floats/scientific notation in CSVs)
-

2) Inputs supported (four modes)

At runtime you choose one of four input modes:

1. **TIC mode:** user provides a TIC ID.
2. **Gaia DR3 source_id mode:** user provides a Gaia DR3 `source_id`; script resolves it to RA/Dec via Gaia TAP.
3. **Coordinate mode:** user provides RA/Dec in degrees.
4. **Batch CSV mode:** user provides a CSV; script infers columns for TIC/Gaia/RA/Dec.

At startup (CLI), the script also asks for a **flux mode**:

- The Default mode: use the default flux given for any sector as given by lightkurve.
- detrended: **SPOC → PDCSAP, QLP → DET_FLUX/KSPSAP_FLUX** (fallback to SAP)
- raw: **SAP everywhere** (fallback to detrended choices if SAP is unavailable)

In batch mode, the same choice is passed through `run_batch_from_csv(..., flux_mode=...)` and stored in the per-target metrics.

Target naming

The script defines a label used in titles and logs:

- TIC mode → "TIC {tic_id}"
- Gaia mode → "Gaia DR3 {source_id}"
- Coordinate mode → "RA={ra}, Dec={dec}"

Output folder naming

Each target is written into its own subfolder using:

- TIC_{tic} if tic is provided
- GAIA_{source_id} if gaia source_id is provided
- otherwise RA_{ra}_DEC_{dec} (six decimals)

This is implemented by `sanitize_target_id()`.

There is an option to save the plots to directories by the plot type instead of the stars, so that the plots of the same type for all stars are in the same folder.

Plots can go to:

- `output_dir/<target_id>/periodogram_and_folded.png (by_star), or`
- `output_dir/periodogram_and_folded/<target_id>.png (by_plot_type)`

3) High-level pipeline (flow)

```
INPUT (TIC or Gaia or RA/Dec)
|
v
SEARCH (Lightkurve) -> SPOC first, else QLP
|
v
DOWNLOAD each sector
|
v
SELECT FLUX COLUMN (author-aware)
|
v
PER-SECTOR CLEANING
- remove NaNs
- remove inf
- normalize
- MAD spike filtering
- drop sector if too many points removed
|
v
OPTIONAL: DROP SCATTER-OUTLIER SECTORS (MAD-of-sectors)
|
v
CONCATENATE sectors
|
v
QUALITY FILTER: keep only QUALITY == 0 for LS
|
v
LOMB-SCARGLE (GLS chi2, standard normalization)
|
v
PEAK PICKING + PERIOD UNCERTAINTY
|
```

4) Search stage (Lightkurve)

Function: `search_tess_lightcurves(...)`

Inputs: - `tic_id` OR `gaia_source_id` OR (`ra`, `dec`)

Implementation notes for Gaia IDs:

- `gaia_source_id` is parsed with a robust integer parser that tolerates strings like "12345.0" or "1.2345e+18" (using Decimal), because large IDs can be mangled by CSV/Excel formatting.
- Resolved Gaia coordinates are cached in-memory (`_GAIA_COORD_CACHE`) during the run to avoid repeated TAP queries for repeated IDs.
- `search_radius` (default 2 arcmin)
- `select_closest_on_sky=True`

Search behavior:

- If `tic_id` is given:
 1. `lk.search_lightcurve("TIC {tic_id}", mission="TESS", author="SPOC")`
 2. If empty → repeat with `author="QLP"`
- If `gaia_source_id` is given and `ra/dec` are not provided:
 - Resolve to coordinates via Gaia TAP:

```
SELECT ra, dec
FROM gaiadr3.gaia_source
WHERE source_id = <sid>
```
- If `ra/dec` are given:
 1. Build `SkyCoord(ra, dec)`.
 2. Search within `radius=search_radius` using SPOC; if empty, try QLP.

Sorting:

- Results are sorted by `sequence_number` (sector) ascending.

“Closest target” option:

- If the search result table contains both `distance` and `target_name`, the code selects the **closest target_name on the sky** and keeps only rows with that `target_name`. This attempts to avoid mixing multiple nearby sources.

5) Download stage + cache corruption handling

Function: `safe_download(sr, max_tries=2)`

This wraps Lightkurve's `.download()`.

If download fails and the error message contains a cached corrupt FITS filename (matched by `_CORRUPT_FITS_RE`), the script:

- deletes the corrupt file from disk
- retries download

This prevents a common failure mode where a partially cached file keeps breaking subsequent runs.

6) Flux column selection (SPOC vs QLP) + `flux_mode`

Function: `pick_flux_columns(lc, author="", flux_mode="default")`

Returns:

- `flux_kind`: one of "PDCSAP", "SAP", "DET_FLUX", "KSPSAP_FLUX", "UNKNOWN"
- `flux_col`: the chosen flux column name (case preserved) or None
- `flux_err_col`: the chosen flux error column name or None

The selection depends on **two things**:

- 1) what Lightcurve product was downloaded (typically from the search table author field: SPOC vs QLP)
- 2) the user-facing choice `flux_mode`.

`flux_mode` behavior

A) `flux_mode="raw"`

Prefer **SAP** if available:

- Uses `sap_flux` / `sap_flux_err` when present.
- If SAP does not exist for that product, it falls back to the *detrended* logic below.

B) `flux_mode="detrended"`

Prefer the usual “detrended” columns for each author:

- **SPOC**: `pdcsap_flux` / `pdcsap_flux_err` (fallback: `sap_flux` / `sap_flux_err`)
- **QLP**: `det_flux` / `det_flux_err` (fallback: `kpsap_flux` / `kpsap_flux_err`, then `sap_flux`)

C) `flux_mode="default"`

The default branch tries to infer which flux family is in use from Lightcurve metadata (`lc.meta["FLUX_ORIGIN"]`) and returns `flux_kind` but typically leaves `flux_col=None`.

As a result, in default mode the pipeline usually does **not** force `lc.select_flux(...)`; it uses whatever Lightcurve has already set as `lc.flux` and records the `flux_kind` decision.

Enforcing the selected column

After `pick_flux_columns(...)`, the script attempts to force the choice (when `flux_col` is not None) via: - `lc = lc.select_flux(flux_col, flux_err_col)` (or without error column)

If `select_flux` fails for any reason, the script continues with the default `lc.flux`.

Recorded metadata

For each successfully downloaded sector, the code stores in `meta`:

- `sector` (`sequence_number`)
- `author`
- `exptime`
- `flux_kind` plus the selected `flux_col` and `flux_err_col` (empty strings if unknown)

The final `cleaned_lightcurve.csv` includes `flux_kind` per cadence (repeated for all cadences within each sector).

7) Per-sector preprocessing

For each downloaded sector light curve:

1. `lc = lc.remove_nans()`
2. Explicit removal of infinities:
 - keeps only cadences where `flux` is finite and (if present) `flux_err` is finite.
3. Force writable arrays:
 - `lc.flux = lc.flux.copy()` and same for `flux_err`
 - this prevents “output array is read-only” errors during later operations.
4. Normalize:
 - `lc = lc.normalize()`

Time system handling

The code expects BTJD (BJD – 2457000).

It applies: - If `median(time) > 10000`, interpret as full BJD and subtract 2457000.

This is a heuristic.

Quality array extraction

The code tries multiple options:

- `lc.quality` if present
- else `lc["quality"]` if available
- else creates zeros (treat as all good)

All quality values are cast to `int64`.

QUALITY decoding + compact summaries (for plot legends and metrics)

The code includes a small QUALITY decoding layer to make flagged points interpretable without expanding full bitmasks in the UI:

- `_tess_quality_strings()` tries to import Lightcurve’s official `TessQualityFlags`.`STRINGS`. If unavailable, it falls back to a minimal (SPOC-ish) bit-value→short-name map.
- A QLP-specific sentinel bit is explicitly added:
 - `2**30 → "QLP quality flag"`

Two derived helpers are then used:

- 1) **Bit prevalence / summaries**
 - `quality_bit_fractions(q, denom=..., ignore_mask=...)` returns a ranked list of which bit-values are present and their fractions.
 - `quality_summary(q, frac_thresh, max_flags, denom, ignore_mask, prefix)` returns compact strings such as:
 - "Flag: Argabrightening 15%, Reaction wheel desat 8%"
- 2) **“Soft-only” summaries** The code defines a set of “hard discard” bit indices (`_HARD_BIT_IDXS`) and forms `HARD_QUALITY_MASK`. When building the *soft* summary line for a sector title, it ignores hard bits:
 - `mi["quality_soft_summary_raw"] = quality_summary(..., ignore_mask=HARD_QUALITY_MASK, prefix="Soft")`

This is intended for *description*, not filtering: **the actual filtering rule remains `QUALITY==0` for the Lomb–Scargle analysis set.**

8) Per-sector spike filtering (MAD-based)

After normalization (so flux is ~1), each sector is filtered using an asymmetric MAD rule.

Definitions per sector:

- `median_flux = median(f_sec)`
- `mad_flux = 1.4826 * median(|f_sec - median_flux|)`

Hard-coded parameters (current code):

- `K_UP = 5.0` (aggressive clipping of upward spikes)
- `K_DOWN = 500.0` (very permissive on downward excursions for including transits)
- `min_flux = 0.4` (absolute floor)

Cadence keep condition:

- `f_sec <= median_flux + K_UP * mad_flux`
- `f_sec >= median_flux - K_DOWN * mad_flux`
- `f_sec >= min_flux`
- `AND time/flux/err finite`

Sector survival requirement

- `KEEP_FRAC_MIN = 0.50`
- If fewer than 50% of points remain in a sector after filtering, that sector is dropped entirely.
- This is designed to remove sectors dominated by artifacts or extreme problems.

Per-sector QUALITY bookkeeping (stored in `meta`)

During this stage the code also computes and stores per-sector counts and flag summaries (used in titles and in the output metrics):

- `n_kept_good / n_kept_flagged` (based on `quality==0`)
- `quality_bits_present_raw / quality_names_present_raw`
- `quality_soft_summary_raw` (soft-only prevalence summary; hard bits ignored)

Interpretation: With `K_DOWN=500`, almost all deep downward deviations will be retained unless they violate `min_flux`. This is a deliberate choice to avoid removing genuine dips, at the cost of retaining more low-flux artifacts.

9) Dropping whole sectors by anomalous scatter

This step triggers only if at least 3 sectors survive the previous stage.

Settings:

- `DROP_OUTLIER_SECTORS = True`
- `SCATTER_METHOD = "mad"`
- `SCATTER_SIGMA_CLIP = 4.0`
- `CLIP_IN_LOG = True`

Computation:

1. For each sector compute a scatter metric:
- MAD-based sigma of flux (or other options: std, p2p)
2. Work in $\log_{10}(\text{scatter})$ (because scatter can vary by orders of magnitude).
3. Compute median and MAD across sectors.

4. Compute a z-score per sector.
5. Drop sectors where $|z| > 4$.

This removes sectors that are far noisier (or sometimes far quieter due to pathological behavior) than the ensemble.

10) Concatenation and the QUALITY==0 analysis set

After sector-level filtering/dropping:

- Concatenate all surviving sectors into arrays:

```
- time_all, flux_all, flux_err_all, qual_all
```

Define “good” cadences: `- good_all = (qual_all == 0)`

Only these cadences are used for Lomb–Scargle:

- `time = time_all[good_all]`
- `flux = flux_all[good_all]`
- `flux_err = flux_err_all[good_all]`

Flagged cadences are retained only for plotting overlays.

11) Lomb–Scargle periodogram configuration

Frequency grid

- Baseline: `T = time.max() - time.min()`

Default (non-interactive) grid:

- `fmin = 0.004 [1/day]`
- `fmax = 4.0 [1/day]`
- `Nfreq = 240,000`
- `freq = linspace(fmin, fmax, Nfreq)`

Interactive mode can override fmin/fmax by user input.

Interpretation:

- This searches periods between ~1 day and ~T days (because `fmin~1/T`).
- The script does not search periods shorter than 1 day by default.

Lomb–Scargle method

Uses `astropy.timeseries.LombScargle`:

- If `flux_err` is nonzero, it is treated as `dy` (weights).
- If `dy` has invalid values (`<=0`, `nan`, `inf`) they are replaced by the median of valid `dy`; if none are valid, `dy` is discarded.

Power is computed using:

- `method="chi2"`
- `normalization="standard"`

This is a GLS-style formulation with a constant offset (`fit_mean` default).

12) Peak selection

A “significant” peak is defined by:

- `threshold = median(power) * 1.2`
- peaks found by `scipy.signal.find_peaks(power, height=threshold)`

Best peak selection:

- If peaks exist: take the highest-power peak among those peaks.
- If none: fall back to global maximum of power.

Derived best period:

- `best_freq = freq[best_idx]`
 - `best_period = 1/best_freq`
-

13) Period uncertainty estimate (FWHM proxy)

A simple “FWHM in frequency” approach is used:

1. Compute `half_max = power[best_idx]/2`.
2. Find nearest indices left and right where power falls below `half_max`.
3. `fwhm = freq[right_idx] - freq[left_idx]`
4. Convert to a period uncertainty:
 - `period_err = fwhm / best_freq^2`

This is not a strict statistical uncertainty; it is a width-based heuristic.

Baseline sanity check

If `best_period > T` (longer than the data baseline), the script rejects it and walks down the sorted list of powers until it finds a period $\leq T$.

14) Diagnostics: noise floor and “non-flat” region

The code computes a high-frequency “flat floor” and a diagnostic maximum frequency where the periodogram is still “non-flat”.

Function: `estimate_flat_floor_and_nonflat_max(freq, power, ...)`

Steps:

- Compute `log10(power)` after clipping power to a small epsilon.
- Smooth log-power with a moving-average window (default window length ~ 2% of array length, min 101, odd).
- Define a high-frequency region as the top 20% of frequencies.
- Estimate:
 - `floor_lp = median(smoothed log-power in high-frequency region)`
 - `floor_mad_lp = MAD scatter around that floor`
 - `thr_lp = floor_lp + k * floor_mad_lp (k=3)`
- Find indices where smoothed log-power exceeds threshold; the largest frequency among them is `f_nonflat_max`.

This is currently used for metrics only.

Function: `focus_fmax_by_excess_power(freq, power, floor_power, q=0.995)`

- Computes “excess power” = $\max(\text{power} - \text{floor_power}, 0)$
 - Picks fmax such that 99.5% of cumulative excess power lies below fmax.
-

15) Plot products

The plotting behavior depends on `plot_by_sector` and `per_sector_periodograms`.

A1) Per-sector plotting (default in batch) – combined periodogram only

Produces two figures:

- 1) `periodogram_and_folded.png`
 - Panel 1: Lomb–Scargle periodogram ($x=\text{frequency}$, $y=\text{power}$)
 - draws vertical line at best frequency
 - draws horizontal threshold line
 - Panel 2: full-data folded light curve at P_{best} (good points only, because the LS arrays are `QUALITY==0`)
 - Panel 3: folded at $2 \times P_{\text{best}}$
- 2) `per_sector_raw_and_folded.png`
 - One row per surviving sector.
 - Column 1: time series
 - good points: black dots
 - flagged points: red x
 - flagged legend label is a compact `quality_summary(...)` computed **relative to the flagged subset**
 - the title can include a second line with a **soft-only QUALITY prevalence** summary (`Soft: ...`) computed with `ignore_mask=HARD_QUALITY_MASK`
 - Column 2: folded at P_{best}
 - Column 3: folded at $2 \times P_{\text{best}}$

Marker sizes automatically shrink for large N using `marker_size_for_panel()`.

A2) Per-sector plotting and Per-sector periodogram

Same as A1, produces two figures:

- 1) `periodogram_and_folded.png` - unchanged
- 2) `per_sector_raw_and_folded.png`
 - One row per surviving sector.
 - Column 1: Same as A1
 - Column 2: periodogram for individual sector
 - Column 3: folded at P_{best} of individual sector
 - Column 4: folded at $2 \times P_{\text{best}}$ of individual sector

B) Non-sector plotting

Produces a single figure:

- `lightcurve_periodogram_folded.png`
 - Includes a full light curve panel, periodogram panel, and folded panels.
-

16) CSV product: `cleaned_lightcurve.csv`

For every target, the script writes: - `<output_dir>/<target_id>/cleaned_lightcurve.csv`

Columns:

- sector : sector number if known, otherwise -1
- author : product author string from search table (e.g., SPOC / QLP)
- flux_kind : one of PDCSAP/SAP/DET_FLUX/KSPSAP_FLUX/UNKNOWN
- time_btjd : time in BTJD (heuristically converted)
- flux_norm : normalized flux (post filtering and concatenation)
- flux_err : flux error (normalized in practice if Lightcurve carried it through normalize)
- quality : TESS quality bitmask (int)
- is_good : 1 if quality==0 else 0

Important: this CSV includes **both good and flagged cadences**, with a label.

17) Metrics dictionary (per target)

When `return_metrics=True`, the function returns a dict with:

Identification

- status, error
- label, target_id
- tic_id, gaia_source_id, ra_deg, dec_deg
- flux_mode_requested

Search / sector bookkeeping

- n_search_results
- n_downloaded_sectors
- n_usable_sectors
- sector_numbers_downloaded (from the search table when available)
- authors_used, flux_kinds_used
- raw_points_per_sector, kept_points_per_sector, kept_frac_per_sector
- sector_ls_json when per_sector_periodograms is used

Concatenated time series stats

- time_min_btjd, time_max_btjd, baseline_days
- n_points_total_all (all cadences after sector filtering, including flagged)
- n_points_good_total, n_points_flagged_total (counts using `quality==0` on concatenated arrays)
- n_points_total (the `QUALITY==0` LS analysis set size)
- flux_mean, flux_std

LS configuration and results

- fmin, fmax, n_freq
- power_min, power_max, power_median
- n_peaks, threshold
- best_idx, best_freq, best_period_days, period_err_days
- T0_btjd

Peak-height diagnostics

- best_power
- best_power_over_median
- power_floor_flat, best_power_over_flat_floor
- power_floor_local, best_power_over_local_floor

“Non-flat” diagnostic region (periodogram structure)

- `f_nonflat_max`
- `floor_log10, floor_mad_log10`

Outputs

- `outdir`
- `cleaned_lightcurve_csv`

Per-sector QUALITY summaries (populated from meta)

The code writes per-sector summary lists into the metrics dict based on the meta entries built during per-sector filtering:

- `good_points_per_sector, flagged_points_per_sector`
- `quality_bits_present_per_sector, quality_names_present_per_sector`
- `quality_soft_summary_per_sector`

These are used to audit which sectors were mostly clean vs dominated by particular flagged conditions, while keeping the LS analysis itself restricted to `QUALITY==0` cadences.

18) Batch mode: `run_batch_from_csv(...)`

Input CSV reading

- CSV is read with `dtype=str` specifically to avoid losing precision in large IDs (important for Gaia DR3 `source_id`).

Column inference

If `tic_col/gaia_col/ra_col/dec_col` are not given, the script guesses them using `infer_columns()`:

- TIC candidates: `tic, tic_id, ticid, tess_tic, id, ...`
- Gaia candidates: `source_id, gaia_source_id, gaia_id, dr3_source_id, ...`
- RA candidates: `ra, ra_deg, radeg`
- Dec candidates: `dec, dec_deg, dedeg, ...`

Target precedence

For each row:

1. Use TIC if present
2. else use Gaia `source_id` if present
3. else use RA/Dec if available

Extra metadata passed into plot titles

If present in the CSV:

- a P_{Rot} from Gao
- a $P/\sin i$ from Gaia
- a M_G from Gaia.

Output

The output is the input CSV plus all metric fields:

- default output name: `<input>_with_TESS_metrics.csv`



19) Parameter reference (defaults)

Flux selection

- `flux_mode = "default" | "detrended" | "raw"`
 - default: use the default flux given for any sector as given by lightkurve. (and typically does not force `select_flux`, see Section 6)
 - detrended: SPOC→PDCSAP, QLP→DET_FLUX/KSPSAP_FLUX (fallback to SAP)
 - raw: prefer SAP everywhere (fallback to detrended choices)

Search

- `search_radius = 2 arcmin`
- authors: SPOC first, then QLP
- `select closest target_name (coordinate searches): True`

Per-sector filtering

- `K_UP = 5.0`
- `K_DOWN = 500.0`
- `min_flux = 0.4`
- `KEEP_FRAC_MIN = 0.50`

Sector outlier dropping

- `method = "mad"`
- `sigma clip = 4.0`
- `clip in log10(scatter) space`

Lomb-Scargle

- Good cadences: `QUALITY == 0`
- `fmin = 0.004`
- `fmax = 4.0`
- `Nfreq = 240000`
- `method="chi2", normalization="standard"`
- `peak threshold = median(power) * 1.2`

Folding

- `T0 = floor(min(time))`
- folds at P and 2P
- harmonic fit: `fn.FitFunction_All_Points(..., N=3)`

20) Reproducibility checklist (for a new user)

How to use my code:

1. Create an environment with the required packages.
 2. Run the script.
 3. Choose input mode and supply target.
 4. Confirm the generated folder exists:
 - `TESSPlots/TIC_<id>/...` or `TESSPlots/GAIA_<sid>/...`
 5. Confirm that the following are created (per target):
 - `periodogram_and_folded.png`
 - `per_sector_raw_and_folded.png` (if per-sector mode)
 - `cleaned_lightcurve.csv`
 - `periodogram.csv.gz`
 - `periodogram.npz`
 - `periodogram_meta.json`
 6. For batch mode, confirm:
 - `<input>_with_TESS_metrics.csv`
-